

chamber_stuff/basic_control.py

```

1  #!/usr/bin/env python3
2  import argparse
3  import atexit
4  import bisect
5  import csv
6  import math
7  import os
8  import signal
9  import sys
10 import threading
11 import time
12 from dataclasses import dataclass
13 from typing import List, Optional, Tuple
14 from adafruit_extended_bus import ExtendedI2C as I2C
15
16 try:
17     import board
18     import adafruit_sht4x
19 except ImportError:
20     board = None
21     adafruit_sht4x = None
22
23 try:
24     import lib8mosind
25 except ImportError:
26     lib8mosind = None
27
28
29 # -----
30 # Hardware mapping
31 # -----
32 PRIMARY_HEATER_CHANNEL = 8
33 ASSIST_HEATER_CHANNEL = 7
34 MISTER_CHANNEL = 1
35 STACK_LEVEL = 0
36 ALL_MOSFET_CHANNELS = range(1, 9)
37
38 # -----
39 # Timing
40 # -----
41 DECISION_INTERVAL_S = 1.0
42 OUTPUT_REASSERT_INTERVAL_S = 0.25
43 LOOP_SLEEP_S = 0.05
44
45 # -----
46 # Mister tuning
47 # -----
48 MISTER_PULSE_ON_S = 0.25
49 MISTER_COOLDOWN_S = 4.0
50
51 # -----

```

```
52 # Humidity tuning
53 # -----
54 HUMIDITY_SETPOINT_OFFSET_RH = 0.0
55 HUMIDITY_DEADBAND_RH = 0.0
56
57 # -----
58 # Heater staging
59 # -----
60 ASSIST_HEATER_RH_THRESHOLD = 65.0
61
62 # -----
63 # Safety settings
64 # -----
65 MAX_CONSECUTIVE_SENSOR_FAILURES = 3
66
67 PLAUSIBLE_TEMP_MIN_C = -20.0
68 PLAUSIBLE_TEMP_MAX_C = 80.0
69 PLAUSIBLE_RH_MIN = 0.0
70 PLAUSIBLE_RH_MAX = 100.0
71
72 SAFETY_MIN_TEMP_C: Optional[float] = None
73 SAFETY_MAX_TEMP_C: Optional[float] = None
74 SAFETY_MIN_RH: Optional[float] = None
75 SAFETY_MAX_RH: Optional[float] = None
76
77 MAX_SKIPPED_DECISION_SECONDS = 5
78
79 # Disk syncing: False is usually better on Pi SD cards for latency/stability.
80 FSYNC_EVERY_ROW = False
81
82 # Diagnostics
83 SLOW_SENSOR_READ_WARN_S = 1.0
84 SLOW_LOG_WRITE_WARN_S = 1.0
85
86 # Watchdog: if the main loop stops heartbeating for this long,
87 # force all outputs off from a background thread.
88 WATCHDOG_TIMEOUT_S = 10.0
89 WATCHDOG_POLL_S = 0.25
90
91
92 @dataclass(frozen=True)
93 class ProgramPoint:
94     time_s: float
95     temp_c: float
96     rh_percent: float
97
98
99 class ProgramSchedule:
100     def __init__(self, points: List[ProgramPoint]):
101         if not points:
102             raise ValueError("Program CSV must contain at least one row.")
103
104         points = sorted(points, key=lambda p: p.time_s)
105
```

```
106     for i in range(len(points) - 1):
107         if points[i + 1].time_s <= points[i].time_s:
108             raise ValueError("Program times must be strictly increasing.")
109
110     self.points = points
111     self.times = [p.time_s for p in points]
112     self.end_time_s = points[-1].time_s
113
114     @classmethod
115     def from_csv(cls, path: str) -> "ProgramSchedule":
116         points = []
117         with open(path, "r", newline="") as f:
118             reader = csv.reader(f)
119             for row_num, row in enumerate(reader, start=1):
120                 if not row or all(not cell.strip() for cell in row):
121                     continue
122
123                 if len(row) != 3:
124                     raise ValueError(
125                         f"Row {row_num} in {path!r} must have exactly 3 columns: "
126                         "time_s, temp_c, rh_percent"
127                     )
128
129                 try:
130                     points.append(
131                         ProgramPoint(
132                             time_s=float(row[0]),
133                             temp_c=float(row[1]),
134                             rh_percent=float(row[2]),
135                         )
136                     )
137                 except ValueError as exc:
138                     raise ValueError(f"Invalid numeric value on row {row_num}: {row}") from exc
139
140         return cls(points)
141
142     def setpoint_at(self, t_s: float) -> Tuple[float, float]:
143         if t_s <= self.points[0].time_s:
144             return self.points[0].temp_c, self.points[0].rh_percent
145
146         if t_s >= self.points[-1].time_s:
147             return self.points[-1].temp_c, self.points[-1].rh_percent
148
149         idx = bisect.bisect_right(self.times, t_s) - 1
150         p0 = self.points[idx]
151         p1 = self.points[idx + 1]
152
153         frac = (t_s - p0.time_s) / (p1.time_s - p0.time_s)
154         temp = p0.temp_c + frac * (p1.temp_c - p0.temp_c)
155         rh = p0.rh_percent + frac * (p1.rh_percent - p0.rh_percent)
156         return temp, rh
157
158
```

```
159 class ChamberIO:
160     def read_sensor(self) -> Tuple[float, float]:
161         raise NotImplementedError
162
163     def set_heaters(self, primary_active: bool, assist_active: bool) -> None:
164         raise NotImplementedError
165
166     def set_mister(self, active: bool) -> None:
167         raise NotImplementedError
168
169     def all_outputs_off(self) -> None:
170         raise NotImplementedError
171
172     def close(self) -> None:
173         pass
174
175
176 class RealChamberIO(ChamberIO):
177     def __init__(
178         self,
179         stack_level: int,
180         primary_heater_channel: int,
181         assist_heater_channel: int,
182         mister_channel: int,
183     ):
184         if board is None or adafruit_sht4x is None:
185             raise RuntimeError(
186                 "Missing sensor libraries. Install adafruit-blinka and "
187                 "adafruit-circuitpython-sht4x."
188             )
189         if lib8mosind is None:
190             raise RuntimeError(
191                 "Missing Sequent library. Install it with:\n"
192                 "    python3 -m pip install SM8mosind"
193             )
194
195         self.stack_level = stack_level
196         self.primary_heater_channel = primary_heater_channel
197         self.assist_heater_channel = assist_heater_channel
198         self.mister_channel = mister_channel
199
200         i2c = I2C(3)
201         self.sht = adafruit_sht4x.SHT4x(i2c)
202         self.sht.mode = adafruit_sht4x.Mode.NOHEAT_HIGHPRECISION
203
204         self.all_outputs_off()
205         atexit.register(self._safe_shutdown_hook)
206
207     def _safe_shutdown_hook(self) -> None:
208         try:
209             self.all_outputs_off()
210         except Exception:
211             pass
212
```

```
213     def read_sensor(self) -> Tuple[float, float]:
214         temperature_c, rh_percent = self.sht.measurements
215         return float(temperature_c), float(rh_percent)
216
217     def _set_channel(self, channel: int, active: bool) -> None:
218         lib8mosind.set(self.stack_level, channel, 1 if active else 0)
219
220     def set_heaters(self, primary_active: bool, assist_active: bool) -> None:
221         self._set_channel(self.primary_heater_channel, primary_active)
222         self._set_channel(self.assist_heater_channel, assist_active)
223
224     def set_mister(self, active: bool) -> None:
225         self._set_channel(self.mister_channel, active)
226
227     def all_outputs_off(self) -> None:
228         for channel in ALL_MOSFET_CHANNELS:
229             try:
230                 self._set_channel(channel, False)
231             except Exception:
232                 pass
233
234     def close(self) -> None:
235         self.all_outputs_off()
236
237
238 class MockChamberIO(ChamberIO):
239     def __init__(self, initial_temp_c: float = 25.0, initial_rh_percent: float =
40.0):
240         self.temp_c = initial_temp_c
241         self.rh_percent = initial_rh_percent
242         self.primary_heater_active = False
243         self.assist_heater_active = False
244         self.mister_active = False
245
246     def read_sensor(self) -> Tuple[float, float]:
247         heater_strength = 0.0
248         if self.primary_heater_active:
249             heater_strength += 0.03
250         if self.assist_heater_active:
251             heater_strength += 0.03
252
253         if heater_strength > 0:
254             self.temp_c += heater_strength
255         else:
256             self.temp_c -= 0.005
257
258         if self.mister_active:
259             self.rh_percent += 0.20
260         else:
261             self.rh_percent -= 0.03
262
263         self.temp_c = max(-20.0, min(80.0, self.temp_c))
264         self.rh_percent = max(0.0, min(100.0, self.rh_percent))
265         return self.temp_c, self.rh_percent
```

```
266
267     def set_heaters(self, primary_active: bool, assist_active: bool) -> None:
268         self.primary_heater_active = primary_active
269         self.assist_heater_active = assist_active
270
271     def set_mister(self, active: bool) -> None:
272         self.mister_active = active
273
274     def all_outputs_off(self) -> None:
275         self.primary_heater_active = False
276         self.assist_heater_active = False
277         self.mister_active = False
278
279     def close(self) -> None:
280         self.all_outputs_off()
281
282
283 class LoopWatchdog:
284     def __init__(self, io: ChamberIO, timeout_s: float, poll_s: float):
285         self.io = io
286         self.timeout_s = timeout_s
287         self.poll_s = poll_s
288         self._lock = threading.Lock()
289         self._last_beat = time.monotonic()
290         self._stop_event = threading.Event()
291         self.tripped = threading.Event()
292         self._thread = threading.Thread(target=self._run, daemon=True)
293
294     def start(self) -> None:
295         self._thread.start()
296
297     def stop(self) -> None:
298         self._stop_event.set()
299         self._thread.join(timeout=1.0)
300
301     def beat(self) -> None:
302         with self._lock:
303             self._last_beat = time.monotonic()
304
305     def _age(self) -> float:
306         with self._lock:
307             return time.monotonic() - self._last_beat
308
309     def _run(self) -> None:
310         while not self._stop_event.wait(self.poll_s):
311             age = self._age()
312             if age > self.timeout_s:
313                 if not self.tripped.is_set():
314                     self.tripped.set()
315                     print(
316                         f"[ERROR] Watchdog tripped after {age:.2f}s without main-
loop heartbeat; forcing all outputs OFF",
317                         file=sys.stderr,
318                         flush=True,
```

```
319         )
320     try:
321         self.io.all_outputs_off()
322     except Exception as exc:
323         print(f"[ERROR] Watchdog OFF command failed: {exc}",
324               file=sys.stderr, flush=True)
325
326 def fsync_file(f) -> None:
327     f.flush()
328     if FSYNC_EVERY_ROW:
329         os.fsync(f.fileno())
330
331
332 def ensure_output_path(path: str, overwrite: bool) -> None:
333     directory = os.path.dirname(os.path.abspath(path))
334     os.makedirs(directory, exist_ok=True)
335     if os.path.exists(path) and not overwrite:
336         raise FileExistsError(
337             f"Refusing to overwrite existing file: {path}\n"
338             f"Use --overwrite if you really want to replace it."
339         )
340
341
342 def derive_events_path(log_csv: str) -> str:
343     base, ext = os.path.splitext(log_csv)
344     if not ext:
345         ext = ".csv"
346     return f"{base}_events{ext}"
347
348
349 def humidity_control_target(program_rh_sp: float) -> float:
350     return program_rh_sp + HUMIDITY_SETPOINT_OFFSET_RH
351
352
353 def humidity_trigger_threshold(program_rh_sp: float) -> float:
354     return humidity_control_target(program_rh_sp) - HUMIDITY_DEADBAND_RH
355
356
357 def assist_heater_allowed(program_rh_sp: float) -> bool:
358     return humidity_control_target(program_rh_sp) >= ASSIST_HEATER_RH_THRESHOLD
359
360
361 def is_plausible_reading(temp_c: float, rh: float) -> bool:
362     return (
363         PLAUSIBLE_TEMP_MIN_C <= temp_c <= PLAUSIBLE_TEMP_MAX_C
364         and PLAUSIBLE_RH_MIN <= rh <= PLAUSIBLE_RH_MAX
365     )
366
367
368 def safety_limit_breached(temp_c: float, rh: float) -> Optional[str]:
369     if SAFETY_MIN_TEMP_C is not None and temp_c < SAFETY_MIN_TEMP_C:
370         return f"Temperature below hard safety limit: {temp_c:.2f} C <
{SAFETY_MIN_TEMP_C:.2f} C"
```

```
371     if SAFETY_MAX_TEMP_C is not None and temp_c > SAFETY_MAX_TEMP_C:
372         return f"Temperature above hard safety limit: {temp_c:.2f} C >
{SAFETY_MAX_TEMP_C:.2f} C"
373     if SAFETY_MIN_RH is not None and rh < SAFETY_MIN_RH:
374         return f"Humidity below hard safety limit: {rh:.2f} % <
{SAFETY_MIN_RH:.2f} %"
375     if SAFETY_MAX_RH is not None and rh > SAFETY_MAX_RH:
376         return f"Humidity above hard safety limit: {rh:.2f} % >
{SAFETY_MAX_RH:.2f} %"
377     return None
378
379
380 def fmt_value(value: Optional[float]) -> str:
381     return "" if value is None else f"{value:.2f}"
382
383
384 def format_csv_row(
385     elapsed_s: int,
386     temp_sp: float,
387     temp_meas: Optional[float],
388     rh_program_sp: float,
389     rh_control_sp: float,
390     rh_meas: Optional[float],
391     primary_heater_active: bool,
392     assist_heater_active: bool,
393     mister_active: bool,
394 ) -> List[object]:
395     return [
396         elapsed_s,
397         f"{temp_sp:.2f}",
398         fmt_value(temp_meas),
399         f"{rh_program_sp:.2f}",
400         f"{rh_control_sp:.2f}",
401         fmt_value(rh_meas),
402         1 if primary_heater_active else 0,
403         1 if assist_heater_active else 0,
404         1 if mister_active else 0,
405     ]
406
407
408 def format_status_line(
409     elapsed_s: int,
410     temp_sp: float,
411     temp_meas: Optional[float],
412     rh_program_sp: float,
413     rh_control_sp: float,
414     rh_meas: Optional[float],
415     primary_heater_active: bool,
416     assist_heater_active: bool,
417     mister_active: bool,
418 ) -> str:
419     temp_meas_txt = " ERR " if temp_meas is None else f"{temp_meas:5.2f}"
420     rh_meas_txt = " ERR " if rh_meas is None else f"{rh_meas:4.2f}"
421
```

```

422     return (
423         f"[T = {elapsed_s:4d}] "
424         f"St = {temp_sp:5.2f}|Mt = {temp_meas_txt} "
425         f"HEAT {1 if primary_heater_active else 0}/{1 if assist_heater_active else
0} "
426         f"|| Sh = {rh_control_sp:4.2f}| Mh = {rh_meas_txt} "
427         f"MIST {1 if mister_active else 0}"
428     )
429
430
431 def print_status_header() -> None:
432     print("[T] St|Mt HEAT H8/H7 || Sh|Mh MIST", flush=True)
433
434
435 def run_program(
436     program_csv: str,
437     log_csv: str,
438     events_csv: str,
439     dry_run: bool = False,
440     overwrite: bool = False,
441 ) -> None:
442     ensure_output_path(log_csv, overwrite=overwrite)
443     ensure_output_path(events_csv, overwrite=overwrite)
444
445     schedule = ProgramSchedule.from_csv(program_csv)
446
447     if dry_run:
448         io = MockChamberIO(
449             initial_temp_c=schedule.points[0].temp_c,
450             initial_rh_percent=schedule.points[0].rh_percent,
451         )
452     else:
453         io = RealChamberIO(
454             stack_level=STACK_LEVEL,
455             primary_heater_channel=PRIMARY_HEATER_CHANNEL,
456             assist_heater_channel=ASSIST_HEATER_CHANNEL,
457             mister_channel=MISTER_CHANNEL,
458         )
459
460     watchdog = LoopWatchdog(io, timeout_s=WATCHDOG_TIMEOUT_S,
poll_s=WATCHDOG_POLL_S)
461     watchdog.start()
462
463     stop_requested = False
464
465     def request_stop(signum, frame):
466         nonlocal stop_requested
467         stop_requested = True
468
469     signal.signal(signal.SIGINT, request_stop)
470     signal.signal(signal.SIGTERM, request_stop)
471     if hasattr(signal, "SIGHUP"):
472         signal.signal(signal.SIGHUP, request_stop)
473     if hasattr(signal, "SIGQUIT"):

```

```
474     signal.signal(signal.SIGQUIT, request_stop)
475
476     primary_heater_cmd = False
477     assist_heater_cmd = False
478
479     mister_pulse_active = False
480     mister_on_until = 0.0
481     mister_cooldown_until = 0.0
482     mister_cmd = False
483
484     consecutive_sensor_failures = 0
485     last_decision_second = -1
486     next_output_reassert_time = 0.0
487     watchdog_event_logged = False
488
489     def emit_event(
490         writer: csv.writer,
491         events_file,
492         elapsed_s: float,
493         level: str,
494         message: str,
495     ) -> None:
496         row = [f"{elapsed_s:.2f}", level, message]
497         writer.writerow(row)
498         fsync_file(events_file)
499         print(f"[{level}] t={elapsed_s:.2f}s {message}", file=sys.stderr,
flush=True)
500
501     try:
502         with open(log_csv, "w", newline="") as log_f, open(events_csv, "w",
newline="") as events_f:
503             log_writer = csv.writer(log_f)
504             events_writer = csv.writer(events_f)
505
506             log_writer.writerow([
507                 "time_from_start_s",
508                 "temp_setpoint_c",
509                 "measured_temp_c",
510                 "humidity_program_setpoint_rh",
511                 "humidity_control_target_rh",
512                 "measured_humidity_rh",
513                 "primary_heater_active",
514                 "assist_heater_active",
515                 "mister_active",
516             ])
517             fsync_file(log_f)
518
519             events_writer.writerow(["time_from_start_s", "level", "message"])
520             fsync_file(events_f)
521
522             start_monotonic = time.monotonic()
523             print_status_header()
524
525             while True:
```

```

526         watchdog.beat()
527         elapsed = time.monotonic() - start_monotonic
528
529         if watchdog.tripped.is_set():
530             if not watchdog_event_logged:
531                 emit_event(events_writer, events_f, elapsed, "ERROR",
"Watchdog tripped; outputs forced OFF")
532                 watchdog_event_logged = True
533                 break
534
535         if mister_pulse_active and elapsed >= mister_on_until:
536             mister_pulse_active = False
537             mister_cmd = mister_pulse_active
538
539         if elapsed >= next_output_reassert_time:
540             io.set_heaters(primary_heater_cmd, assist_heater_cmd)
541             io.set_mister(mister_cmd)
542             next_output_reassert_time = elapsed + OUTPUT_REASSERT_INTE-
RVAL_S
543
544         if stop_requested:
545             emit_event(events_writer, events_f, elapsed, "INFO", "Stop
requested by signal")
546             break
547
548         if elapsed > schedule.end_time_s:
549             emit_event(events_writer, events_f, elapsed, "INFO", "Program
complete")
550             break
551
552         current_second = int(math.floor(elapsed))
553
554         if current_second <= last_decision_second:
555             time.sleep(LOOP_SLEEP_S)
556             continue
557
558         if last_decision_second >= 0 and current_second -
last_decision_second > MAX_SKIPPED_DECISION_SECONDS:
559             emit_event(
560                 events_writer,
561                 events_f,
562                 elapsed,
563                 "WARN",
564                 f"Control loop delayed; skipped from
{last_decision_second}s to {current_second}s"
565             )
566
567         sample_time = float(current_second)
568         temp_sp, rh_program_sp = schedule.setpoint_at(sample_time)
569         rh_control_sp = humidity_control_target(rh_program_sp)
570         rh_trigger_sp = humidity_trigger_threshold(rh_program_sp)
571
572         measured_temp_c: Optional[float] = None
573         measured_rh: Optional[float] = None

```

```

574
575         try:
576             t_read_start = time.monotonic()
577             measured_temp_c, measured_rh = io.read_sensor()
578             t_read_end = time.monotonic()
579             read_duration = t_read_end - t_read_start
580
581             if read_duration > SLOW_SENSOR_READ_WARN_S:
582                 emit_event(
583                     events_writer,
584                     events_f,
585                     elapsed,
586                     "WARN",
587                     f"Slow sensor read: {read_duration:.2f}s"
588                 )
589
590             if stop_requested:
591                 emit_event(events_writer, events_f, elapsed, "INFO", "Stop
requested after sensor read")
592                 break
593
594             if watchdog.tripped.is_set():
595                 if not watchdog_event_logged:
596                     emit_event(events_writer, events_f, elapsed, "ERROR",
"Watchdog tripped after sensor read; outputs forced OFF")
597                     watchdog_event_logged = True
598                 break
599
600             if not is_plausible_reading(measured_temp_c, measured_rh):
601                 raise ValueError(
602                     f"Implausible sensor reading: T={measured_temp_c:.2f}
C, RH={measured_rh:.2f} %"
603                 )
604             except Exception as exc:
605                 consecutive_sensor_failures += 1
606
607                 primary_heater_cmd = False
608                 assist_heater_cmd = False
609                 mister_pulse_active = False
610                 mister_cmd = False
611                 io.set_heaters(False, False)
612                 io.set_mister(False)
613
614                 emit_event(
615                     events_writer,
616                     events_f,
617                     elapsed,
618                     "WARN" if consecutive_sensor_failures <
MAX_CONSECUTIVE_SENSOR_FAILURES else "ERROR",
619                     f"Sensor read failed ({consecutive_sensor_f-
ailures}/{MAX_CONSECUTIVE_SENSOR_FAILURES}): {exc}"
620                 )
621
622                 if not stop_requested and not watchdog.tripped.is_set():

```

```
623         log_writer.writerow(  
624             format_csv_row(  
625                 current_second,  
626                 temp_sp,  
627                 None,  
628                 rh_program_sp,  
629                 rh_control_sp,  
630                 None,  
631                 False,  
632                 False,  
633                 False,  
634             )  
635         )  
636         t_log_start = time.monotonic()  
637         fsync_file(log_f)  
638         t_log_end = time.monotonic()  
639  
640         log_duration = t_log_end - t_log_start  
641         if log_duration > SLOW_LOG_WRITE_WARN_S:  
642             emit_event(  
643                 events_writer,  
644                 events_f,  
645                 elapsed,  
646                 "WARN",  
647                 f"Slow log write/fsync: {log_duration:.2f}s"  
648             )  
649  
650         print(  
651             format_status_line(  
652                 current_second,  
653                 temp_sp,  
654                 None,  
655                 rh_program_sp,  
656                 rh_control_sp,  
657                 None,  
658                 False,  
659                 False,  
660                 False,  
661             ),  
662             flush=True,  
663         )  
664  
665         last_decision_second = current_second  
666  
667         if consecutive_sensor_failures >= MAX_CONSECUTIVE_SENS-  
OR_FAILURES:  
668             emit_event(events_writer, events_f, elapsed, "ERROR", "Too  
many consecutive sensor failures; aborting")  
669             break  
670  
671         time.sleep(LOOP_SLEEP_S)  
672         continue  
673  
674         if consecutive_sensor_failures > 0:
```

```

675         emit_event(events_writer, events_f, elapsed, "INFO", "Sensor
recovered")
676         consecutive_sensor_failures = 0
677
678         limit_message = safety_limit_breached(measured_temp_c,
measured_rh)
679         if limit_message is not None:
680             primary_heater_cmd = False
681             assist_heater_cmd = False
682             mister_pulse_active = False
683             mister_cmd = False
684             io.set_heaters(False, False)
685             io.set_mister(False)
686
687         emit_event(events_writer, events_f, elapsed, "ERROR",
limit_message)
688
689         if not stop_requested and not watchdog.tripped.is_set():
690             log_writer.writerow(
691                 format_csv_row(
692                     current_second,
693                     temp_sp,
694                     measured_temp_c,
695                     rh_program_sp,
696                     rh_control_sp,
697                     measured_rh,
698                     False,
699                     False,
700                     False,
701                 )
702             )
703             t_log_start = time.monotonic()
704             fsync_file(log_f)
705             t_log_end = time.monotonic()
706
707             log_duration = t_log_end - t_log_start
708             if log_duration > SLOW_LOG_WRITE_WARN_S:
709                 emit_event(
710                     events_writer,
711                     events_f,
712                     elapsed,
713                     "WARN",
714                     f"Slow log write/fsync: {log_duration:.2f}s"
715                 )
716
717             print(
718                 format_status_line(
719                     current_second,
720                     temp_sp,
721                     measured_temp_c,
722                     rh_program_sp,
723                     rh_control_sp,
724                     measured_rh,
725                     False,

```

```

726             False,
727             False,
728         ),
729         flush=True,
730     )
731
732     last_decision_second = current_second
733     break
734
735     temp_demand = measured_temp_c < temp_sp
736     primary_heater_cmd = temp_demand
737     assist_heater_cmd = temp_demand and assist_heater_allowe-
d(rh_program_sp)
738
739     if (
740         (not mister_pulse_active)
741         and (elapsed >= mister_cooldown_until)
742         and (measured_rh < rh_trigger_sp)
743     ):
744         mister_pulse_active = True
745         mister_on_until = elapsed + MISTER_PULSE_ON_S
746         mister_cooldown_until = mister_on_until + MISTER_COOLDOWN_S
747         emit_event(
748             events_writer,
749             events_f,
750             elapsed,
751             "INFO",
752             f"Mister pulse started; ON until {mister_on_until:.2f}s,
cooldown until {mister_cooldown_until:.2f}s"
753         )
754
755         mister_cmd = mister_pulse_active
756
757         if stop_requested:
758             emit_event(events_writer, events_f, elapsed, "INFO", "Stop
requested before output apply")
759             break
760
761         if watchdog.tripped.is_set():
762             if not watchdog_event_logged:
763                 emit_event(events_writer, events_f, elapsed, "ERROR",
"Watchdog tripped before output apply; outputs forced OFF")
764                 watchdog_event_logged = True
765             break
766
767         io.set_heaters(primary_heater_cmd, assist_heater_cmd)
768         io.set_mister(mister_cmd)
769
770         if stop_requested:
771             emit_event(events_writer, events_f, elapsed, "INFO", "Stop
requested before main log write")
772             break
773
774         if watchdog.tripped.is_set():

```

```
775         if not watchdog_event_logged:
776             emit_event(events_writer, events_f, elapsed, "ERROR",
777 "Watchdog tripped before main log write; outputs forced OFF")
778             watchdog_event_logged = True
779             break
780     log_writer.writerow(
781         format_csv_row(
782             current_second,
783             temp_sp,
784             measured_temp_c,
785             rh_program_sp,
786             rh_control_sp,
787             measured_rh,
788             primary_heater_cmd,
789             assist_heater_cmd,
790             mister_cmd,
791         )
792     )
793     t_log_start = time.monotonic()
794     fsync_file(log_f)
795     t_log_end = time.monotonic()
796
797     log_duration = t_log_end - t_log_start
798     if log_duration > SLOW_LOG_WRITE_WARN_S:
799         emit_event(
800             events_writer,
801             events_f,
802             elapsed,
803             "WARN",
804             f"Slow log write/fsync: {log_duration:.2f}s"
805         )
806
807     print(
808         format_status_line(
809             current_second,
810             temp_sp,
811             measured_temp_c,
812             rh_program_sp,
813             rh_control_sp,
814             measured_rh,
815             primary_heater_cmd,
816             assist_heater_cmd,
817             mister_cmd,
818         ),
819         flush=True,
820     )
821
822     last_decision_second = current_second
823     time.sleep(LOOP_SLEEP_S)
824
825 finally:
826     try:
827         watchdog.stop()
```

```
828         except Exception:
829             pass
830
831         try:
832             io.all_outputs_off()
833         finally:
834             io.close()
835
836         print("\nSafety shutdown: all MOSFET channels forced OFF.", flush=True)
837
838
839 def parse_args() -> argparse.Namespace:
840     parser = argparse.ArgumentParser(
841         description="Run an environmental chamber program from a CSV file."
842     )
843     parser.add_argument(
844         "program_csv",
845         help="Input CSV: time_from_start_s,temp_c,rh_percent"
846     )
847     parser.add_argument(
848         "--log-csv",
849         default="chamber_log.csv",
850         help="Main 1 Hz log CSV"
851     )
852     parser.add_argument(
853         "--events-csv",
854         default=None,
855         help="Events/warnings CSV (default: derived from --log-csv)"
856     )
857     parser.add_argument(
858         "--overwrite",
859         action="store_true",
860         help="Allow overwriting existing log files"
861     )
862     parser.add_argument(
863         "--dry-run",
864         action="store_true",
865         help="Run without hardware using a simulated chamber"
866     )
867     return parser.parse_args()
868
869
870 if __name__ == "__main__":
871     args = parse_args()
872     events_csv = args.events_csv or derive_events_path(args.log_csv)
873
874     try:
875         run_program(
876             program_csv=args.program_csv,
877             log_csv=args.log_csv,
878             events_csv=events_csv,
879             dry_run=args.dry_run,
880             overwrite=args.overwrite,
881         )
```

```
882 |     except Exception as exc:  
883 |         print(f"\nERROR: {exc}", file=sys.stderr)  
884 |         sys.exit(1)
```